

# Cohesiq: system architecture and design

How the influencer-matching platform is built, from containers and code layout to the matching engine, deal lifecycle and data model. Checked against the source code.

Date 2026-09-24      Schema Alembic 0022      Status As built      Figures 10

## CONTENTS

|                                      |                            |
|--------------------------------------|----------------------------|
| 1. Introduction                      | 7. Matching engine         |
| 2. System architecture               | 8. Deal lifecycle          |
| 3. Backend design                    | 9. Data model              |
| 4. Frontend design                   | 10. Configuration          |
| 5. Authentication and access control | 11. Operations and quality |
| 6. End-to-end workflow               | 12. Known gaps and risks   |

# 1 Introduction

## 1.1 Purpose

This document describes how Cohesiq is built: its runtime topology, code structure, authentication, matching engine, deal lifecycle, data model, configuration and operations. It is written for engineers who will change the system and for reviewers who need to check that it does what the product specification says.

It describes the system *as built*. Every statement was checked against the source code on 2026-09-24 (Alembic migration head `0022`). Where the code and the product spec ( `docs/srs.md` ) differ, the code wins here and the difference is either listed in [§12](#) or already recorded in `docs/plan.md` §3.

## 1.2 The system in one paragraph

Cohesiq is a B2B influencer-matching marketplace for the Bangladesh market. A **brand** briefs a campaign by voice, PDF or text. A deterministic **matching engine** filters and scores every available **creator** and stores the top 10, with an LLM-written explanation for the top 5. The brand shortlists and sends offers; brand and creator negotiate turn by turn; acceptance activates a **contract** that tracks the draft, review, revision and publication of the content, then live metrics and mutual reviews. An **admin** role moderates users, campaigns and reviews.

## 1.3 Glossary

| TERM             | MEANING                                                                                                                                                                                                                                         |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Campaign         | A brand's demand: niche, platforms, budget per creator, deliverables, audience targets. Status: <code>draft</code> · <code>active</code> · <code>in_progress</code> · <code>completed</code> · <code>cancelled</code> · <code>archived</code> . |
| Application      | One creator's participation in one campaign ( <code>campaign_applications</code> ). Carries the deal status from apply/invite to completion.                                                                                                    |
| Negotiation turn | One offer or counter-offer with a rate and terms ( <code>negotiation_turns</code> ). Only the other party's latest turn can be accepted.                                                                                                        |
| Contract         | The agreed deal for one application: payment, deliverables, revision cap, content URLs.                                                                                                                                                         |
| Rate card        | A creator's price for one deliverable on one platform, in BDT ( <code>creator_rate_cards</code> ).                                                                                                                                              |
| Match score      | Weighted 0–1 fit between a campaign and a creator, stored with its six sub-scores in <code>ai_match_scores</code> .                                                                                                                             |

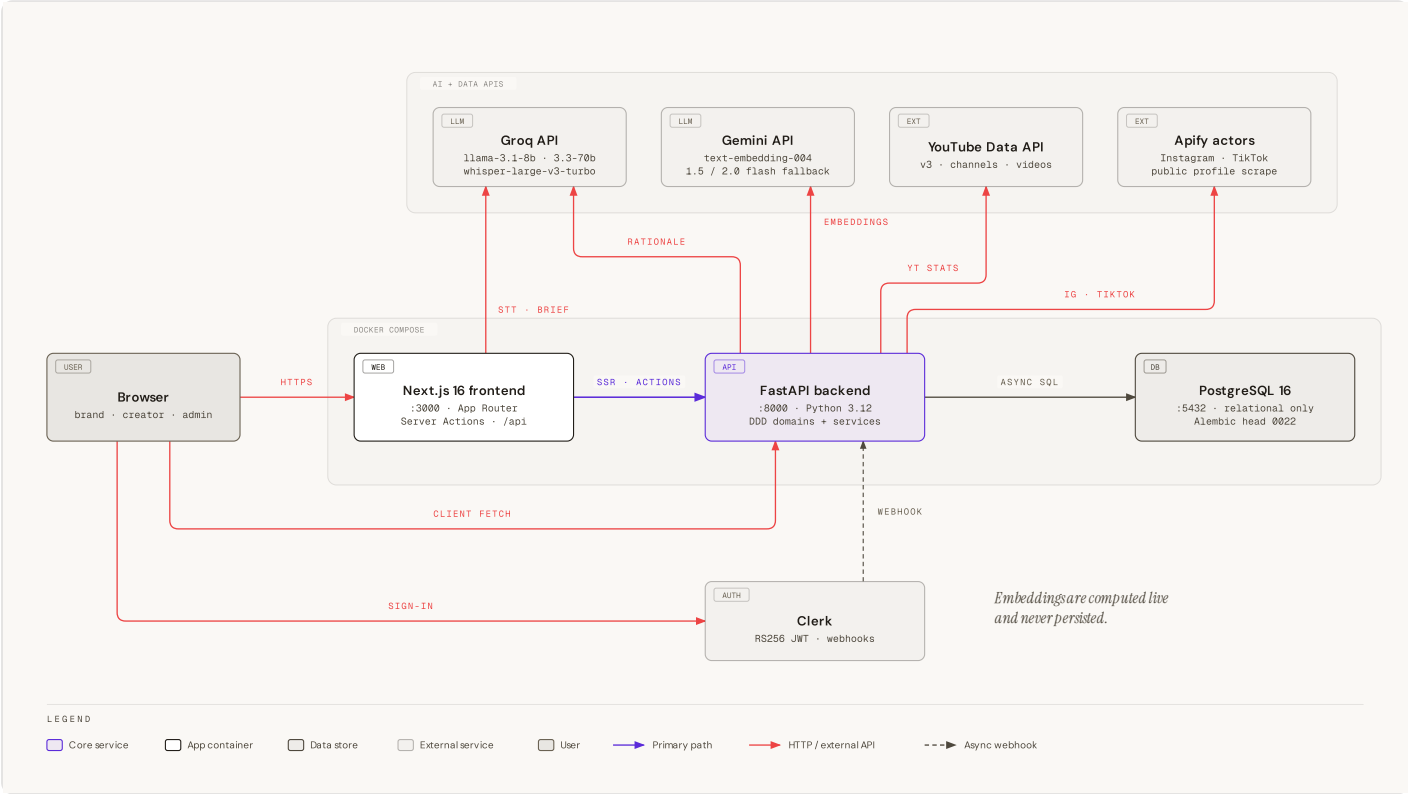
| TERM            | MEANING                                                                                                |
|-----------------|--------------------------------------------------------------------------------------------------------|
| Semantic rescue | Using embedding similarity to give a creator a capped niche score when the exact niche does not match. |
| BDT             | Bangladeshi taka; all prices and payments are stored in BDT.                                           |

## 1.4 Document map

| IF YOU NEED...                      | READ                                                              |
|-------------------------------------|-------------------------------------------------------------------|
| The product requirements            | <code>docs/srs.md</code> , <code>docs/user-stories.md</code>      |
| Why the build differs from the spec | <code>docs/plan.md</code> §3 Divergence Ledger                    |
| Every column of every table         | <code>docs/schema.md</code>                                       |
| Request bodies for each endpoint    | <code>docs/api-testing.md</code> , <code>docs/openapi.json</code> |
| Matching engine internals           | <code>docs/matching-engine.md</code>                              |
| Seeding and resets                  | <code>docs/seeding.md</code>                                      |

## 2 System architecture

Cohesiq runs as three containers under Docker Compose: a Next.js frontend, a FastAPI backend and PostgreSQL. An optional fourth container, ngrok, exposes the frontend publicly when started with `--profile tunnel`. Everything else is an external managed service.



**Figure 1.** Runtime topology. Red arrows are HTTP calls to services outside the Compose network.

### 2.1 Containers

| CONTAINER        | IMAGE / STACK                                                  | PORT | RESPONSIBILITY                                                                                                    |
|------------------|----------------------------------------------------------------|------|-------------------------------------------------------------------------------------------------------------------|
| frontend         | Next.js 16 App Router, React 19, Tailwind v4, shadcn/ui, Clerk | 3000 | All UI; Server Components fetch data; route handlers for speech-to-text, AI suggestions, OAuth and image proxying |
| backend          | FastAPI, Python 3.12, SQLAlchemy 2.0 async (asyncpg)           | 8000 | All business rules, matching, persistence, external data enrichment                                               |
| postgres         | postgres:16                                                    | 5432 | Single relational store. Health-checked with <code>pg_isready</code> ; the backend waits for it                   |
| ngrok (optional) | ngrok/ngrok                                                    | 4040 | Public tunnel to <code>frontend:3000</code>                                                                       |

## 2.2 Request paths

The frontend reaches the backend two ways, and the two base URLs must never be swapped:

| CALLER                                                  | VARIABLE            | LOCAL VALUE           | WHY                                                       |
|---------------------------------------------------------|---------------------|-----------------------|-----------------------------------------------------------|
| Server Components,<br>Server Actions, route<br>handlers | BACKEND_API_URL     | http://backend:8000   | Docker-internal<br>hostname; never<br>reaches the browser |
| Client Components<br>(browser)                          | NEXT_PUBLIC_API_URL | http://localhost:8000 | Must be publicly<br>reachable from the<br>user's machine  |

Both paths go through `fetchApi()` in `lib/api/client.ts`, which attaches the Clerk session token as a bearer token. There are no websockets: the negotiation drawer polls every 4 seconds while open, and application and contract lists poll every 30–90 seconds (`hooks/use-polling`).

## 2.3 External services

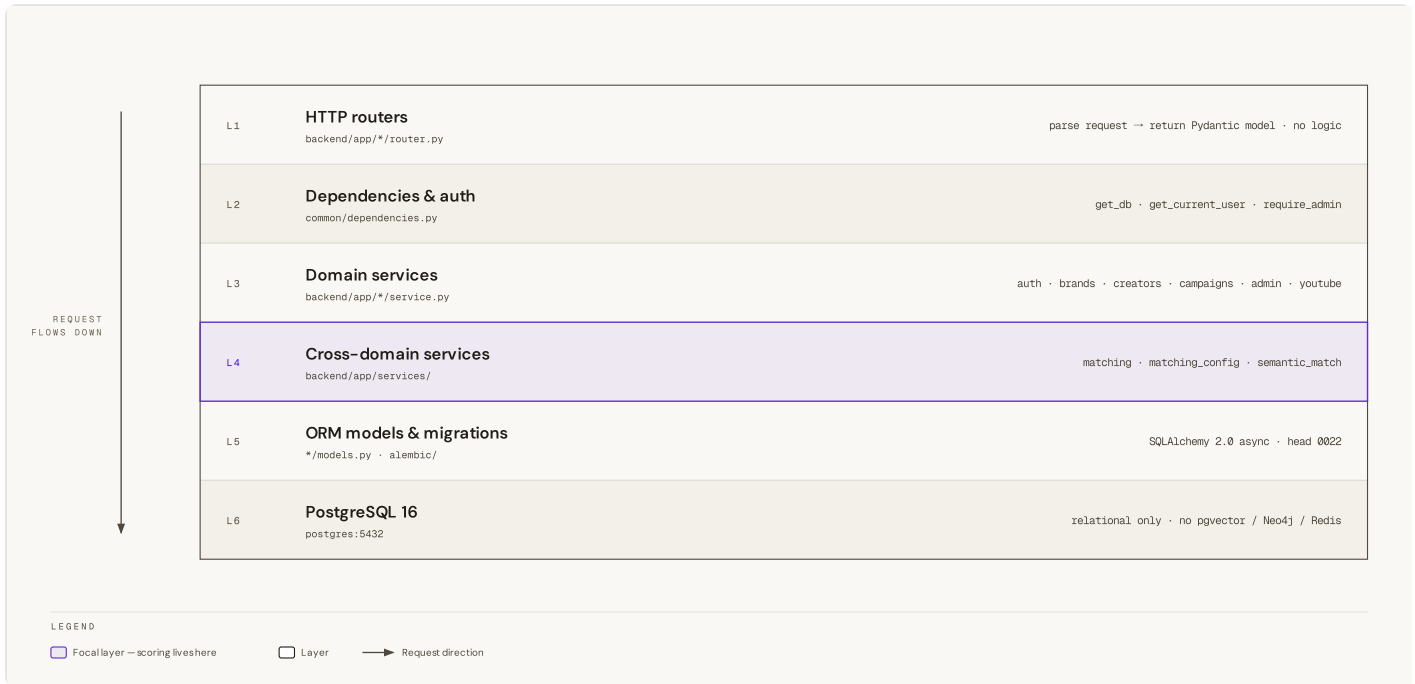
| SERVICE                                  | CALLED FROM                                               | USED FOR                                                        | IF UNAVAILABLE                                                    |
|------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------|
| Clerk                                    | Browser, Next.js, FastAPI                                 | Sign-in, session JWTs,<br>user webhooks, role<br>metadata       | No sign-in; gated routes<br>redirect                              |
| Groq <code>whisper-large-v3-turbo</code> | Next.js <code>/api/transcribe</code>                      | Voice brief → text                                              | Voice briefing fails; text<br>and PDF still work                  |
| Groq <code>llama-3.3-70b</code>          | Next.js <code>/api/campaign-suggestion</code>             | Campaign field<br>suggestions                                   | No suggestions                                                    |
| Groq <code>llama-3.1-8b-instant</code>   | Next.js brief analysis;<br>FastAPI matching               | Brief → form fields; match<br>rationale for the top 5           | Brief: Gemini 2.0 Flash<br>fallback. Rationale:<br>heuristic text |
| Gemini <code>text-embedding-004</code>   | FastAPI matching                                          | Semantic rescue similarity                                      | Token Jaccard similarity                                          |
| YouTube Data<br>API v3                   | FastAPI <code>youtube/</code> ,<br><code>creators/</code> | Channel stats, recent<br>videos → social profile +<br>portfolio | Enrichment returns an<br>error; stored data<br>unchanged          |
| Apify actors                             | FastAPI<br><code>social_ingestion/</code>                 | Public Instagram and<br>TikTok profile stats                    | Enrichment returns an<br>error                                    |

| SERVICE                | CALLED FROM                       | USED FOR                                                   | IF UNAVAILABLE                              |
|------------------------|-----------------------------------|------------------------------------------------------------|---------------------------------------------|
| YouTube / TikTok OAuth | Next.js <code>/api/oauth/*</code> | Creator proves account ownership, then triggers enrichment | Creator can still add the platform manually |

PDF briefs are parsed in the browser with `pdfjs-dist` ; the file never reaches the server. Embeddings are computed per request and never stored. pgvector, Neo4j, Redis and TimescaleDB are deliberately not part of the stack yet (plan §3 D1–D5).

## 3 Backend design

The backend is organised by domain. Each domain folder under `backend/app/` holds `router.py`, `service.py`, `schemas.py` and `models.py`. Routers only parse input and return Pydantic models; all rules live in services. Scoring code shared across domains lives in `services/`.



**Figure 2.** Backend layers. A request flows down; cross-domain scoring (L4) is called from the campaigns service.

### 3.1 Domains and API surface

| PREFIX                  | DOMAIN    | MAIN ENDPOINTS                                                                                                                                                                                     | ACCESS                       |
|-------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| <code>/auth</code>      | auth      | GET <code>/me</code> , POST <code>/onboarding</code> (local <code>/register</code> , <code>/login</code> for HS256 dev tokens)                                                                     | Signed-in                    |
| <code>/brands</code>    | brands    | List, get, <code>/me</code> , update; a brand's campaigns and applications                                                                                                                         | Mixed; writes are owner-only |
| <code>/creators</code>  | creators  | Profile CRUD; platforms (manual + <code>/platforms/{youtube,instagram,tiktok}/enrich</code> ); rate cards; portfolio; collaboration history                                                        | Public reads; owner writes   |
| <code>/campaigns</code> | campaigns | Campaign CRUD and status; apply, invite, respond-<br>invite, shortlist; offer, negotiate, accept, decline; <code>run-matching</code> , <code>matches</code> ; contract actions; analytics; reviews | Role-checked per action      |
| <code>/youtube</code>   | youtube   | <code>/search</code> , <code>/videos/{id}</code> , <code>/channels</code> , <code>/channels/enrichment</code> — stateless read wrapper                                                             | Signed-in                    |

| PREFIX    | DOMAIN   | MAIN ENDPOINTS                                                                     | ACCESS         |
|-----------|----------|------------------------------------------------------------------------------------|----------------|
| /admin    | admin    | Stats; list/deactivate/delete users; moderate campaign status; list/delete reviews | require_admin  |
| /webhooks | webhooks | POST /clerk                                                                        | Svix signature |
| /health   | —        | Liveness                                                                           | Public         |

Three cross-domain reads are mounted directly in `main.py`: a creator's own applications and the public review lists for a creator and a brand. `docs/api-testing.md` has request bodies for every route.

### 3.2 Persistence

SQLAlchemy 2.0 in async mode over `asyncpg`. `get_db` yields one `AsyncSession` per request. Schema changes go through Alembic; the chain is numbered `0001` → `0022` plus a few hash-named revisions. Users and profiles are soft-deleted with `deleted_at`.

## 4 Frontend design

### 4.1 Route groups

| GROUP                | ROUTES                                                                                                                                                   | WHO                              |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| (public)             | / landing, /design-system                                                                                                                                | Anyone                           |
| (auth)               | /onboarding, brand profile, creator personal info → niches → platforms                                                                                   | Signed in, onboarding incomplete |
| (dashboards)/brand   | Dashboard; campaigns (new, detail, edit, matches, rate benchmark, ROI calculator); creators (browse, detail, compare); collaborations; messages; profile | Brand                            |
| (dashboards)/creator | Dashboard; campaigns (browse, detail); collaborations; contracts; messages; profile; connect YouTube / TikTok                                            | Creator                          |
| (admin)              | /admin, users, campaigns, reviews                                                                                                                        | Admin                            |
| api/                 | transcribe, campaign-suggestion, image-proxy, oauth/{youtube,tiktok}/{authorize,callback}                                                                | Route handlers                   |

### 4.2 Rendering pattern

Each `page.tsx` is an async Server Component that only fetches data. Interactive UI lives in a colocated `_components/*Client.tsx` marked "use client". Filters are driven by the URL: the client island calls `router.push('?filter=...')` and the server page re-reads `searchParams`. Server Actions sit in colocated `_actions/` folders, never in a shared top-level folder.

### 4.3 Design system

All colours, type sizes, spacing and shadows come from CSS custom properties in `frontend/design/cohesiq.css`, exposed to Tailwind through the `@theme inline` block in `app/globals.css`. Headings use Plus Jakarta Sans, UI text uses DM Sans. The diagrams in this document use the same palette (brand violet `#5B2BD9`, warm neutrals).

---

## 5 Authentication and access control

### 5.1 Identity flow

1. **Sign-in.** Clerk hosts sign-in. Its session JWT carries `metadata.role ( brand · creator · admin )` and `metadata.onboardingComplete`.
2. **Route gating (frontend).** `proxy.ts` runs Clerk middleware. Public routes skip auth. Unauthenticated users on gated routes go to sign-in; users who haven't finished onboarding go to `/onboarding`; brands are kept out of creator dashboards and vice versa; `/admin` requires `role = admin`.
3. **Token check (backend).** `get_current_user` verifies the RS256 JWT against Clerk's JWKS (fetched once and cached) and the configured issuer, then loads the `users` row by `clerk_id`.
4. **Lazy provisioning.** If no row exists yet (for example after a dev database reset), the backend fetches the user from the Clerk API with `CLERK_SECRET_KEY` and creates it with the real email and role.
5. **Webhooks.** `POST /webhooks/clerk` verifies the Svix signature with `CLERK_WEBHOOK_SECRET`, rejects timestamps older than 5 minutes, and handles `user.created`, `user.updated` and `user.deleted`.

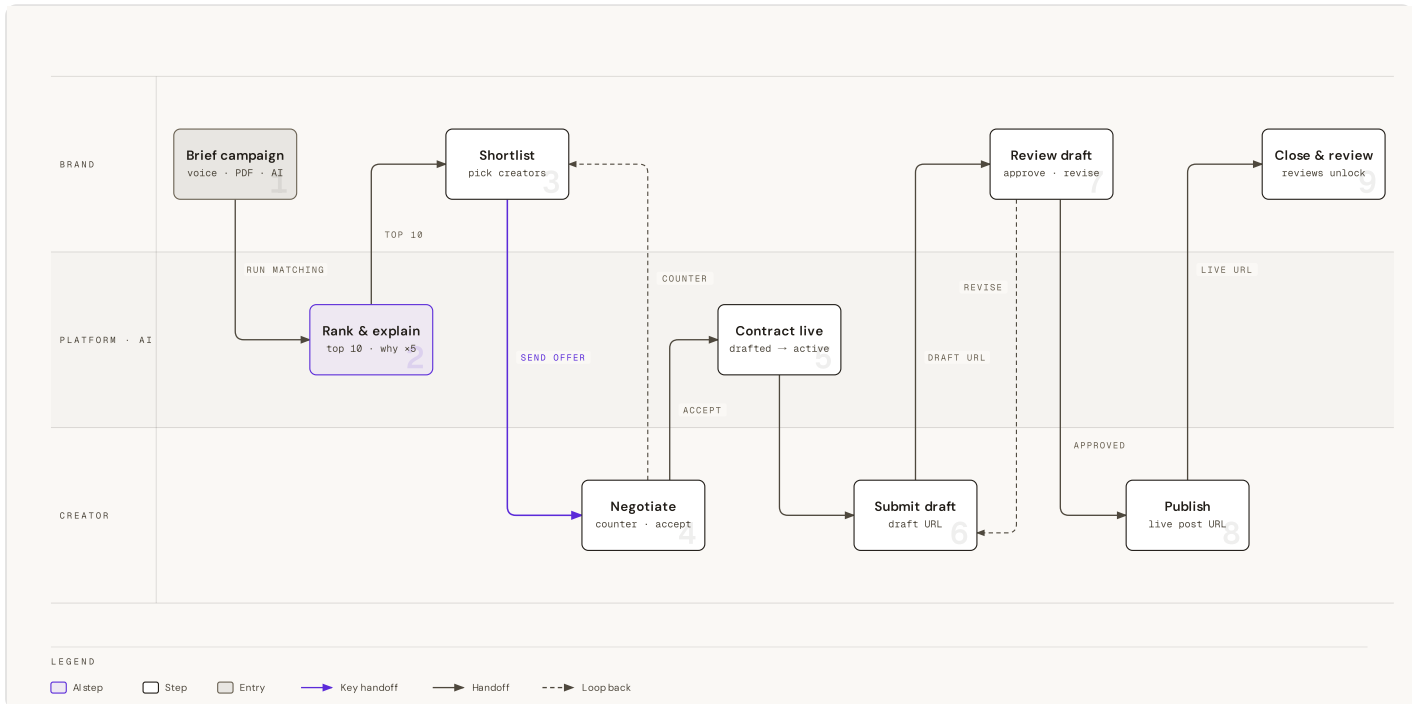
When `CLERK_ISSUER_URL` is unset, the backend accepts local HS256 tokens signed with `SECRET_KEY` instead. That mode exists for development and tests only.

### 5.2 Authorisation

Role checks happen in dependencies and services, not in the UI alone. Brand actions resolve the caller's `brand_profiles` row and compare it with `campaign.brand_id` or `contract.brand_id`; creator actions do the same with `creator_id`. Admin endpoints use `require_admin`, which reads the role from the JWT claims.

## 6 End-to-end workflow

A campaign moves across three actors. The two loops, counter-offers and content revisions, are where most of the elapsed time goes.

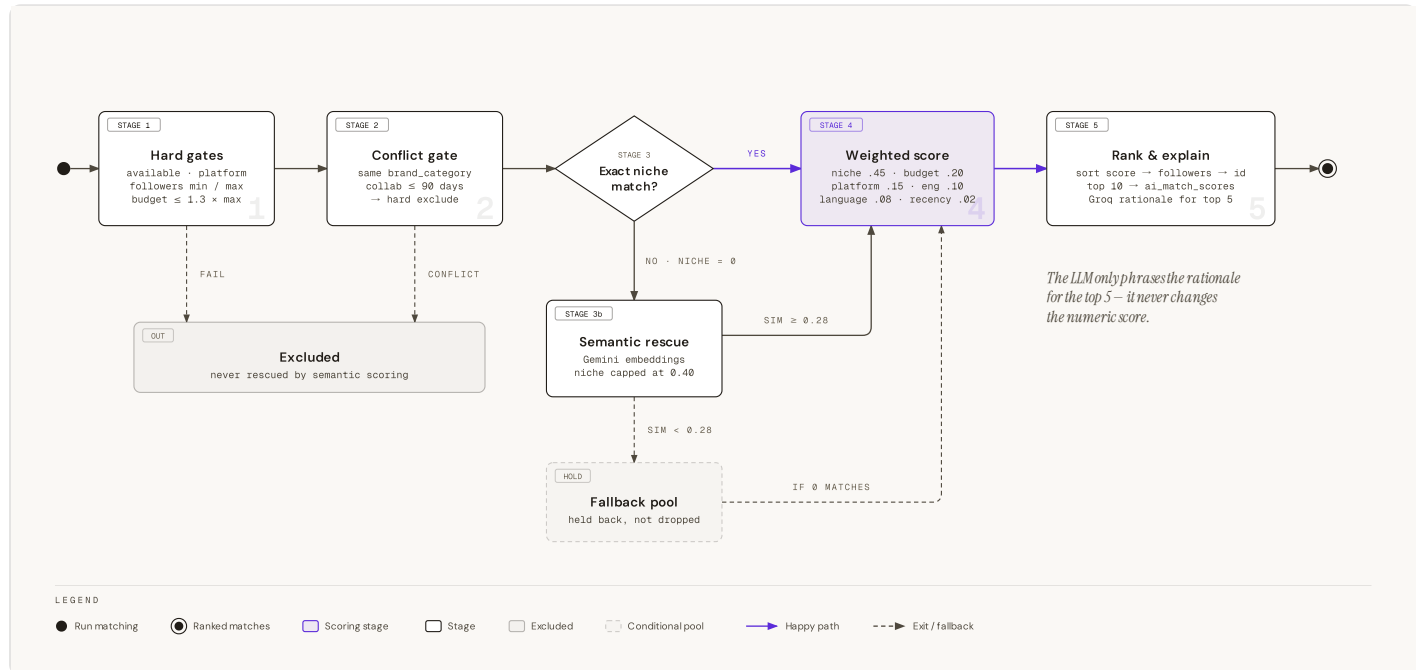


**Figure 3.** Brief to review. Numbers give the order of steps; dashed arrows loop back.

Creators can also enter at step 3: they browse public campaigns and apply, which creates a **pending** application, or they accept a brand's invite. Campaign visibility controls whether a campaign appears in the public browse list.

## 7 Matching engine

POST /campaigns/{id}/run-matching ranks every available creator for one campaign. The engine is deterministic: the same data always gives the same ranking. The LLM only writes the explanation text.

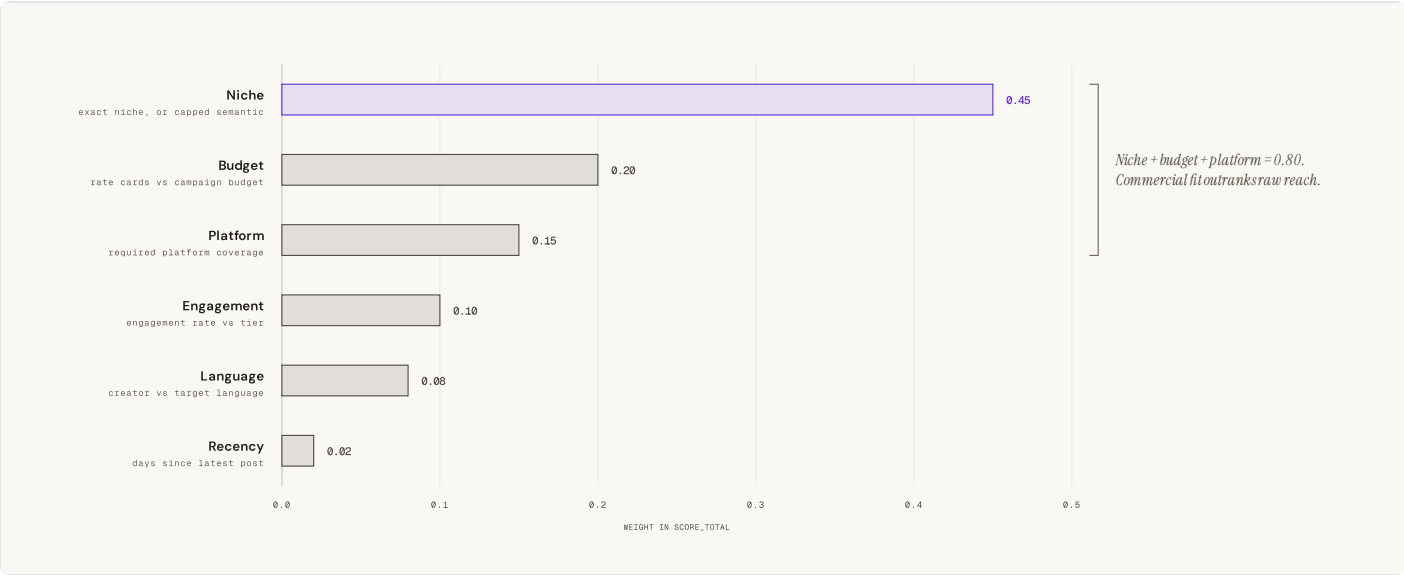


**Figure 4.** The five stages. Hard filters are never undone; semantic rescue can only raise a zero niche score, and only up to 0.40.

### 7.1 Scoring

Each creator that passes stages 1–3 gets six sub-scores between 0 and 1, combined with fixed weights:

$$\text{score\_total} = 0.45 \cdot \text{niche} + 0.20 \cdot \text{budget} + 0.15 \cdot \text{platform} + 0.10 \cdot \text{engagement} + 0.08 \cdot \text{language} + 0.02 \cdot \text{recency}$$



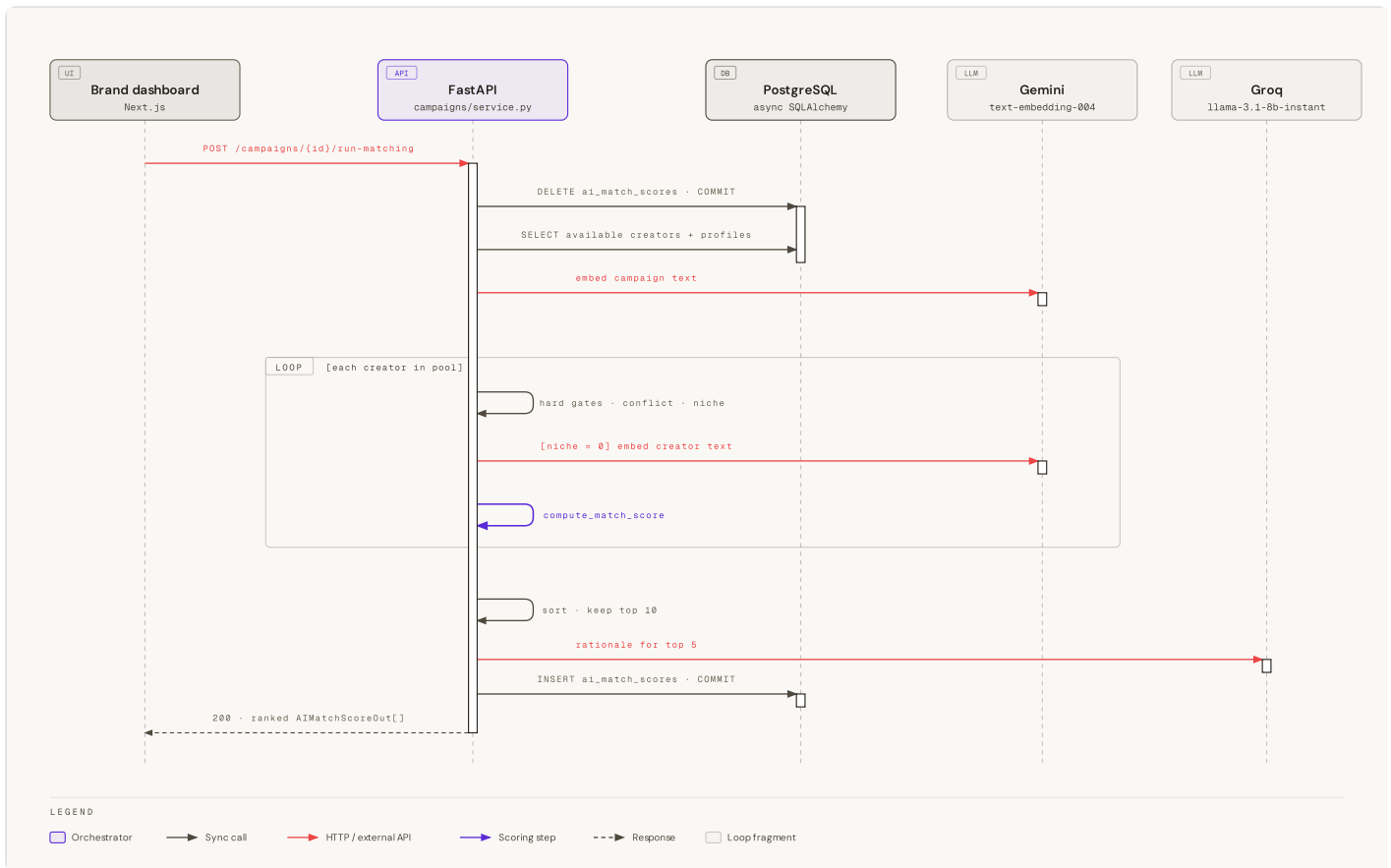
**Figure 5.** Weights from `SCORE_WEIGHTS` . Niche, budget and platform together make up 0.80 of the score.

7.2 Configuration constants

All in `backend/app/services/matching_config.py` ; nothing else defines them.

| CONSTANT                      | VALUE                             | EFFECT                                                                                |
|-------------------------------|-----------------------------------|---------------------------------------------------------------------------------------|
| SCORE_WEIGHTS                 | .45 / .20 / .15 / .10 / .08 / .02 | Weights in the formula above                                                          |
| BUDGET_SOFT_BUFFER_RATIO      | 0.30                              | Creators up to 1.3 × the campaign's max budget per creator still pass the budget gate |
| BUDGET_UNKNOWN_SCORE          | 0.5                               | Budget score when no rate card or tier estimate exists                                |
| SEMANTIC_SIMILARITY_THRESHOLD | 0.28                              | Minimum similarity for a semantic rescue                                              |
| SEMANTIC_RESCUE_NICHE_CAP     | 0.40                              | Highest niche score a rescue can give                                                 |
| UNKNOWN_RECENCY_SCORE         | 0.20                              | Recency score with no post dates                                                      |
| CONFLICT_LOOKBACK_DAYS        | 90                                | Window for the competitor–conflict gate                                               |
| TOP_MATCH_LIMIT               | 10                                | Matches stored per campaign                                                           |
| LLM_RATIONALE_TOP_N           | 5                                 | Matches that get an LLM–written rationale                                             |

## 7.3 One matching run



**Figure 6.** Calls made during one request. Returns are omitted except the final response.

Rerunning replaces the previous result: the campaign's old `ai_match_scores` rows are deleted first. Ties are broken by follower count, then creator id. If strict matching finds nobody, the creators that were excluded *only* by the niche filter are scored instead, so a brand never sees an empty list just because of how the niche was worded.

# 8 Deal lifecycle

The deal is tracked on two records with separate status fields. The **application** covers everything up to agreement; the **contract** covers delivery. Campaign status is not part of the deal flow; the one link is that offers can only be sent while the campaign is `active`.

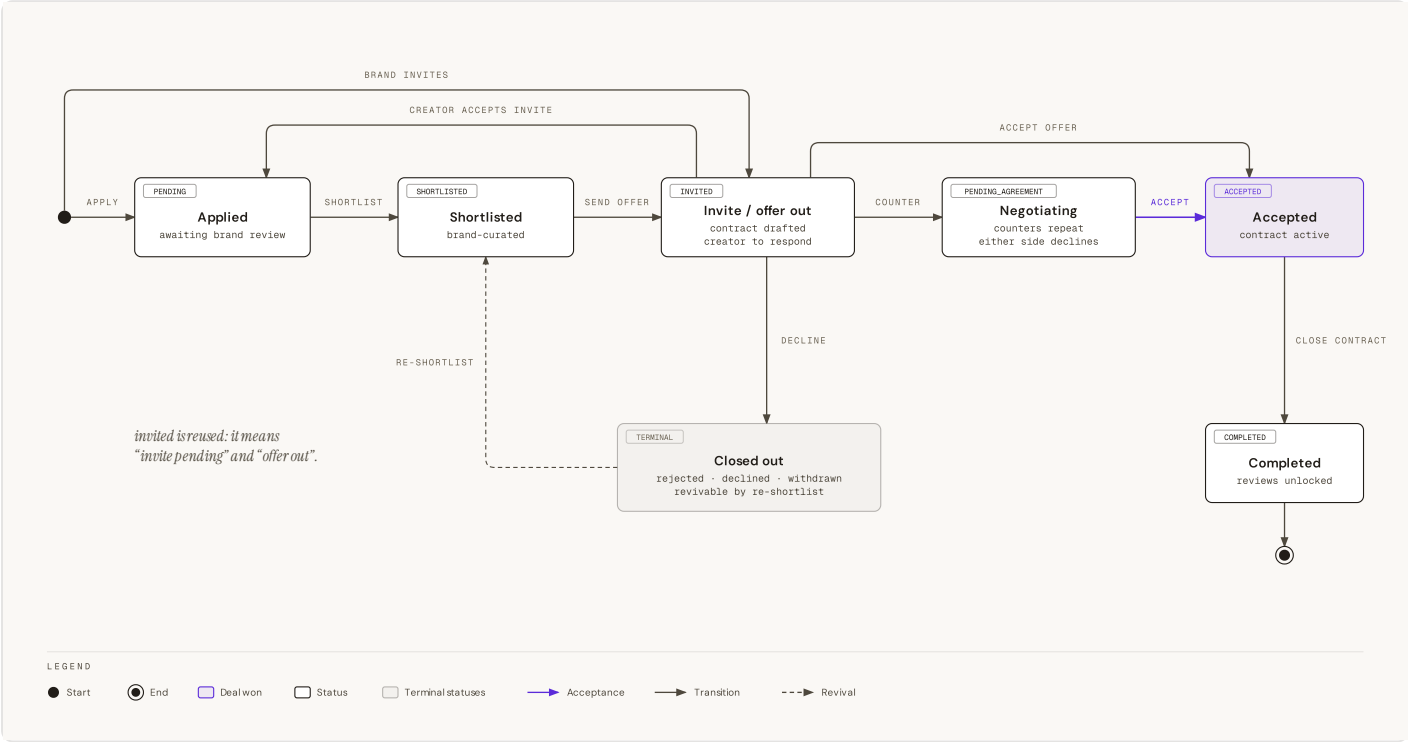


Figure 7. `campaign_applications.status`.

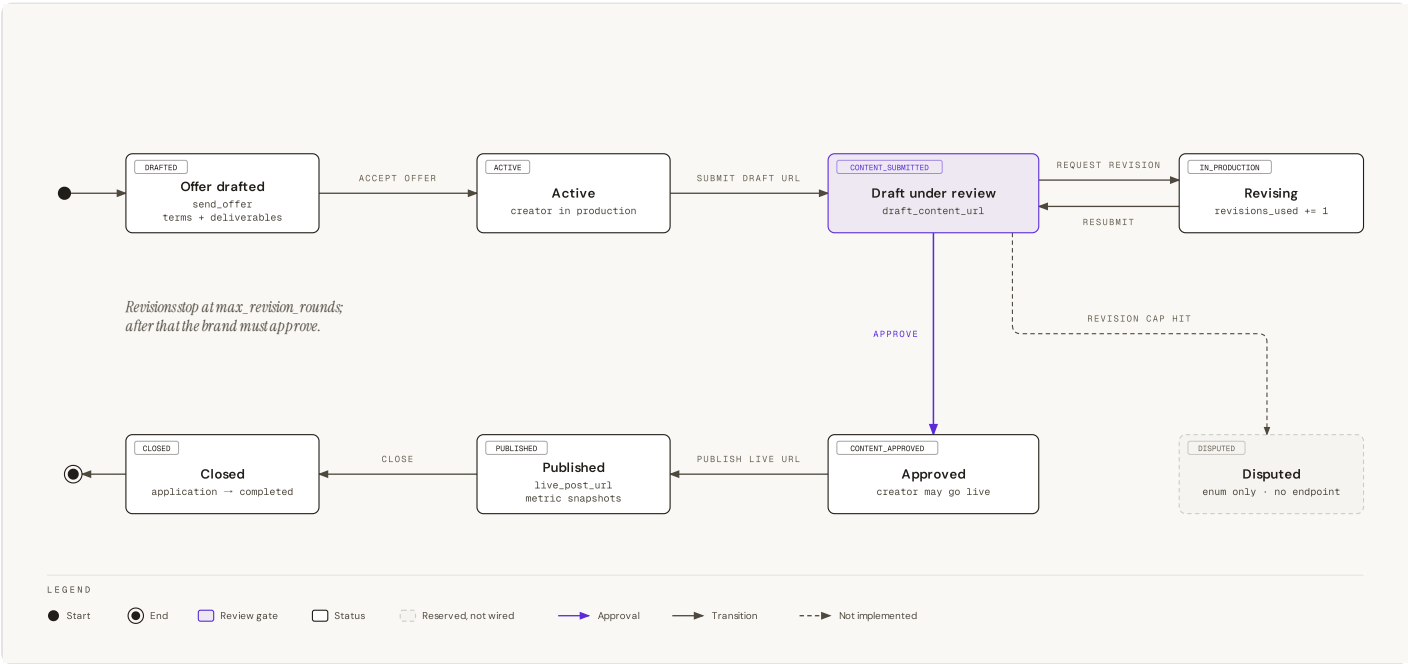


Figure 8. `contracts.status`.

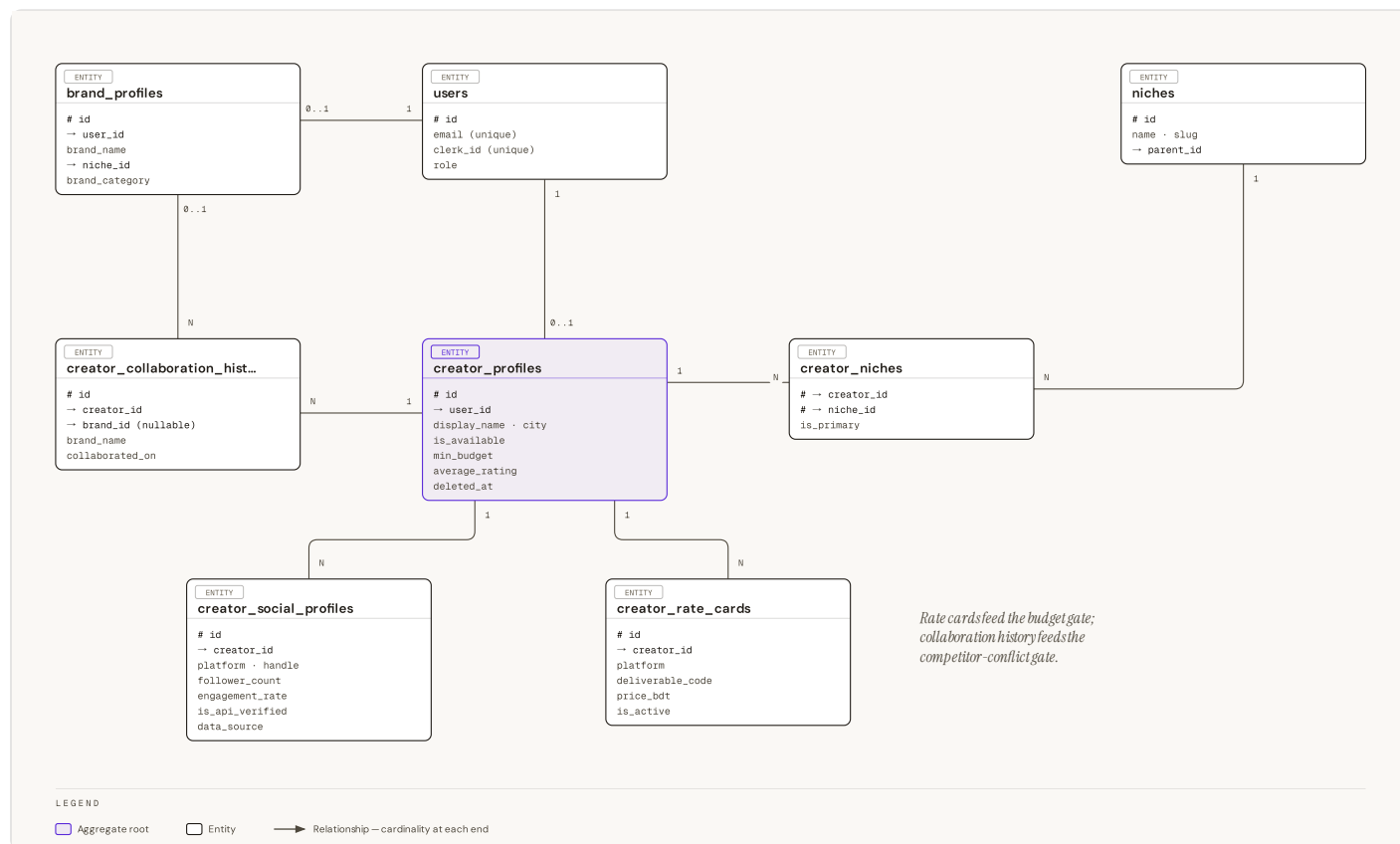
## 8.1 Endpoint to transition map

| ENDPOINT ( UNDER /CAMPAIGNS )              | ACTOR   | APPLICATION                                     | CONTRACT                                         |
|--------------------------------------------|---------|-------------------------------------------------|--------------------------------------------------|
| POST /{id}/apply                           | Creator | → pending                                       | —                                                |
| POST /{id}/invite                          | Brand   | → invited                                       | —                                                |
| POST /{id}/applications/{a}/respond-invite | Creator | invited → pending   declined                    | —                                                |
| POST /{id}/shortlist                       | Brand   | pending   terminal → shortlisted                | —                                                |
| POST /{id}/applications/{a}/offer          | Brand   | shortlisted   pending → invited                 | → drafted                                        |
| POST /{id}/applications/{a}/negotiate      | Either  | invited   pending_agreement → pending_agreement | terms updated                                    |
| POST .../offer/accept                      | Either  | → accepted                                      | drafted → active                                 |
| POST .../offer/decline                     | Either  | → rejected (brand)   declined (creator)         | —                                                |
| PATCH /contracts/{c}/submit-draft          | Creator | —                                               | active   in_production → content_submitted       |
| PATCH /contracts/{c}/request-revision      | Brand   | —                                               | content_submitted → in_production (if under cap) |
| PATCH /contracts/{c}/approve               | Brand   | —                                               | content_submitted → content_approved             |
| PATCH /contracts/{c}/publish               | Creator | —                                               | content_approved → published                     |
| PATCH /contracts/{c}/close                 | Brand   | → completed                                     | published → closed                               |

After closing, both sides can post a review ( POST /campaigns/reviews/ ). Live metrics can be synced while a contract is published or closed; each sync stores a live\_content\_metric\_snapshots row.

## 9 Data model

PostgreSQL 16, relational only. The two diagrams below split the model into supply (who creators are and what they charge) and demand (campaigns and the deals they produce). They show the columns that drive behaviour; `docs/schema.md` lists every column.



**Figure 9.** Identity and creator supply. # = primary key, → = foreign key.

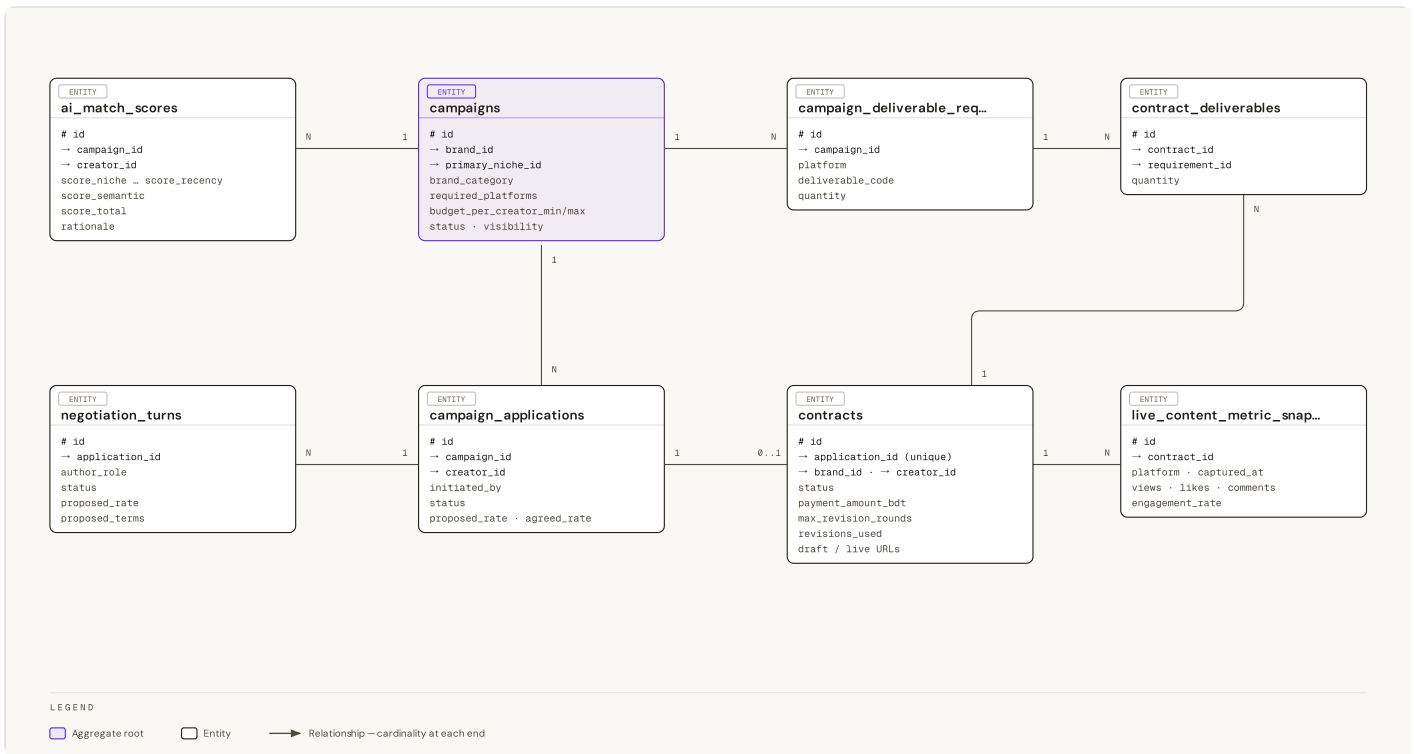


Figure 10. Campaigns and deals.

## 9.1 Integrity rules worth knowing

- One contract per application: `contracts.application_id` is unique.
- `ai_match_scores` is derived data. It is safe to delete; the next matching run rebuilds it.
- `creator_collaboration_history.brand_id` is nullable, so past work with brands that are not on the platform can be recorded. Those rows have no `brand_category` and cannot trigger a competitor conflict.
- `creator_social_profiles.data_source` and `is_api_verified` record where each set of stats came from (API, scrape or self-reported).
- `campaigns.campaign_type` (what the brand wants) and a creator's collaboration types (what the creator offers) are separate lists on purpose. Do not merge them (plan §3.1).

## 10 Configuration

Every variable is listed in the matching `.env.example`. New variables must be added there in both `frontend/` and `backend/` where they apply.

### 10.1 Backend ( `backend/.env` )

| VARIABLE                                                       | PURPOSE                                                                                              |
|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>DATABASE_URL</code>                                      | asyncpg DSN; Compose sets it to the <code>postgres</code> service                                    |
| <code>SECRET_KEY</code>                                        | HS256 signing for local dev tokens                                                                   |
| <code>CLERK_ISSUER_URL</code>                                  | Enables RS256 Clerk verification; set in Compose                                                     |
| <code>CLERK_SECRET_KEY</code>                                  | Lazy user provisioning from the Clerk API                                                            |
| <code>CLERK_WEBHOOK_SECRET</code>                              | Svix signature check on <code>/webhooks/clerk</code> (not yet in <code>.env.example</code> , see G9) |
| <code>GROQ_API_KEY</code>                                      | Match rationale                                                                                      |
| <code>GEMINI_API_KEY</code>                                    | Embeddings for semantic rescue                                                                       |
| <code>YOUTUBE_API_KEY</code>                                   | YouTube Data API v3                                                                                  |
| <code>APIFY_API_TOKEN</code> ,<br><code>APIFY*_ACTOR_ID</code> | Instagram / TikTok enrichment                                                                        |
| <code>NGROK_AUTHTOKEN</code>                                   | Optional tunnel                                                                                      |

### 10.2 Frontend ( `frontend/cohesiq-v0/.env` )

| VARIABLE                                                                          | PURPOSE                                                          |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------|
| <code>NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY</code> ,<br><code>CLERK_SECRET_KEY</code> | Clerk SDK                                                        |
| <code>BACKEND_API_URL</code>                                                      | Server-side base URL ( <code>http://backend:8000</code> )        |
| <code>NEXT_PUBLIC_API_URL</code>                                                  | Browser base URL; must be publicly reachable in each environment |
| <code>GROQ_API_KEY</code> , <code>GEMINI_API_KEY</code>                           | Speech-to-text, brief analysis, suggestions                      |

| VARIABLE                              | PURPOSE                                                |
|---------------------------------------|--------------------------------------------------------|
| YOUTUBE_CLIENT_ID/SECRET/REDIRECT_URI | YouTube OAuth connect                                  |
| TIKTOK_CLIENT_KEY/SECRET/REDIRECT_URI | TikTok OAuth connect                                   |
| NEXT_ALLOWED_DEV_ORIGINS              | Extra hosts allowed to use the dev server's hot reload |

## 11 Operations and quality

### 11.1 Running

| TASK                       | COMMAND                                                                                  |
|----------------------------|------------------------------------------------------------------------------------------|
| Start everything           | <code>docker compose up --build</code>                                                   |
| With public tunnel         | <code>docker compose --profile tunnel up</code>                                          |
| Health                     | <code>docker compose ps</code> , <code>GET /health</code>                                |
| Seed real creator data     | <code>docker compose exec -T postgres psql -U cohesiq -d cohesiq &lt; db/seed.sql</code> |
| Wipe data (keeps schema)   | <code>docker compose exec backend python -m scripts.reset_db</code>                      |
| Assign roles to test users | <code>docker compose exec backend python -m scripts.sync_clerk_users</code>              |

In Compose, both the frontend and the backend mount the source folder and run in dev mode (`pnpm dev`, `uvicorn --reload`). The backend entrypoint runs `alembic upgrade head` before starting, so migrations apply on every container start. `db/seed.sql` is a symlink to the latest snapshot in `db/snapshots/`, holding real YouTube, Instagram and TikTok creator data.

### 11.2 Tests

| SUITE ( <code>BACKEND/TESTS/</code> )                 | TESTS | COVERS                                                                   |
|-------------------------------------------------------|-------|--------------------------------------------------------------------------|
| <code>test_matching_engine.py</code>                  | 7     | Budget buffer, recency buckets, semantic rescue cap, score normalisation |
| <code>test_campaign_application_gatekeeper.py</code>  | 7     | Application status transition guards                                     |
| <code>test_creator_youtube_enrichment.py</code>       | 8     | YouTube → social profile + portfolio persistence                         |
| <code>test_social_ingestion_service.py</code>         | 7     | Apify Instagram / TikTok parsing                                         |
| <code>test_creator_public_social_enrichment.py</code> | 3     | Public enrichment endpoints                                              |
| <code>test_youtube_service.py</code>                  | 3     | YouTube API wrapper                                                      |

Run with `docker compose exec backend python -m unittest discover tests -v`. The frontend has no automated tests; `pnpm run lint` is the only check.

## 12 Known gaps and risks

Found while checking this document against the code. None of these block the current demo, but each one should be settled before production.

| #  | AREA         | FINDING                                                                                                                                                                                          | SUGGESTED ACTION                                                               |
|----|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| G1 | Contracts    | <code>disputed</code> is in the status enum and the revision-limit error tells brands to raise a dispute, but no endpoint sets it.                                                               | Add a dispute endpoint, or remove the status and change the error text.        |
| G2 | Applications | <code>invited</code> means both "invite waiting for the creator" and "offer sent".                                                                                                               | Split into two statuses, or rely on whether a contract exists and document it. |
| G3 | Security     | CORS allows every origin together with credentials ( <code>main.py</code> ).                                                                                                                     | Restrict <code>allow_origins</code> per environment.                           |
| G4 | Performance  | The Gemini embedding call, the Clerk JWKS fetch and lazy provisioning are blocking calls inside async handlers.                                                                                  | Move them to a thread pool or async HTTP clients.                              |
| G5 | Performance  | Matching embeds creator text on every run; nothing is cached.                                                                                                                                    | Persist embeddings (Phase-E pgvector, plan §3) or cache per profile version.   |
| G6 | Docs         | The baseline table in <code>docs/tasks/tasks-navid.md</code> still lists niche .35 / budget .30.                                                                                                 | Update to .45 / .20.                                                           |
| G7 | Quality      | No frontend tests; no end-to-end test of the deal lifecycle.                                                                                                                                     | Add an API-level lifecycle test from offer to close.                           |
| G8 | Deployment   | Compose runs dev servers with source mounts; there is no production build profile.                                                                                                               | Add a production Compose profile or image targets.                             |
| G9 | Config       | <code>CLERK_WEBHOOK_SECRET</code> is required by the webhook router but missing from both <code>.env.example</code> files; the root file also carries misspelt <code>APPIFY_*</code> duplicates. | Add the variable; drop the duplicates.                                         |

Generated by `docs/diagrams/html/techdoc.py` from the figures in `build.py` · `diagram-design skill` · cohesiq profile. Rebuild with `build.py`, then `export_pdf.py` for the PDF.